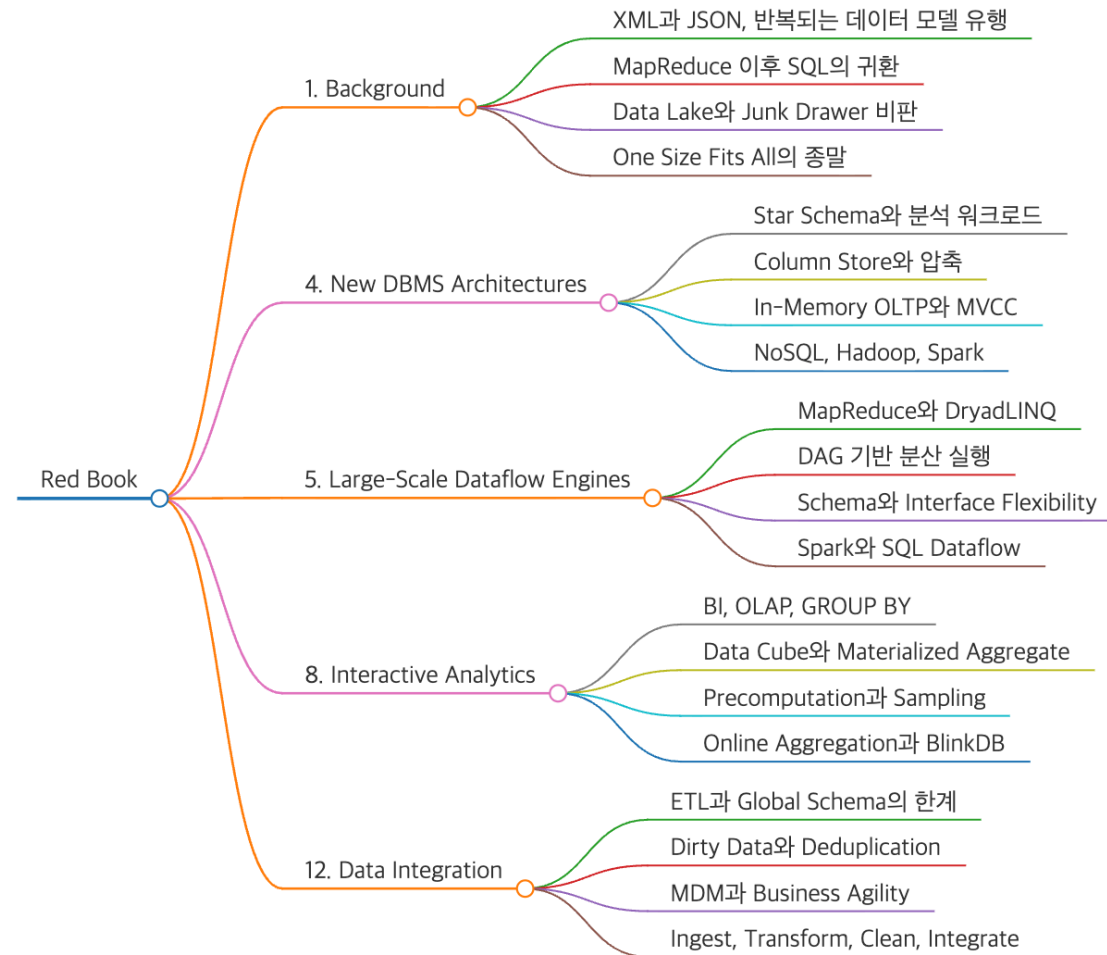


분석계란 무엇인가?

아무도 역사로부터 무엇인가를 배우는 것 같지가 않다

(Kimball Data Warehouse Toolkit, Red Book)

Red Book



전체 12개 장 가운데 분석 시스템과 직접 연결되는 1장, 4장, 5장, 8장, 12장을 선택했습니다.

오늘의 흐름

1. Red Book은 무엇을 비판했는가?
2. 운영 데이터를 어떻게 분석 모델로 바꾸는가?
3. 첫 번째 Star Schema 다음에는 무엇이 필요한가?

"Nobody ever seems to learn anything from history."

"아무도 역사에서 무엇인가를 배우는 것 같지 않다."

Michael Stonebraker, Red Book 5판, Chapter 1: Background

XML과 JSON

"The net-net was that relational won and XML lost."

"I fully expect RDBMSs to subsume JSON as merely a data type (among many) in their systems."

"결국 관계형 모델이 이기고 XML은 졌다. JSON도 RDBMS의 여러 데이터 타입 가운데 하나로 흡수될 것이다."

Michael Stonebraker, [Chapter 1: Background](<http://www.redbook.io/ch1-background.html>)

MapReduce

"Map-Reduce is not an architecture with any broad scale applicability."

"MapReduce는 광범위하게 적용할 수 있는 아키텍처가 아니다."

Michael Stonebraker, [Chapter 1: Background](<http://www.redbook.io/ch1-background.html>)

Data Lake

"Hence, the data lake is just a 'junk drawer' for files in the meantime."

"그러므로 그동안 데이터 레이크는 파일을 넣어두는 잡동사니 서랍일 뿐이다."

Michael Stonebraker, [Chapter 1: Background](<http://www.redbook.io/ch1-background.html>)

One Size Fits All

"It is hard to imagine that 'one size fits all' will ever be the dominant architecture again."

"As such, they deserve to be sent to the 'home for retired software'."

"하나의 구조가 모든 용도에 맞는 시대가 다시 지배적이기는 어렵다. 기존 시스템은 은퇴한 소프트웨어의 집으로 보내야 한다."

Michael Stonebraker, [Chapter 1: Background](<http://www.redbook.io/ch1-background.html>)

분석 시스템의 흐름

시기	주요 흐름	해결하려던 문제
1960년대에서 1970년대	DSS (Decision Support System, 의사결정지원시스템)의 발전	운영 기록을 의사결정에 활용
1980년대	Data Warehouse (데이터 웨어하우스) 개념의 구체화	여러 원천의 장기 이력을 통합
1990년대	EDW (Enterprise Data Warehouse, 전사 데이터 웨어하우스), Data Mart, OLAP (Online Analytical Processing, 온라인 분석 처리)의 확산	부서별 탐색과 다차원 집계
2000년대	MPP (Massively Parallel Processing, 대규모 병렬 처리)와 Column Store의 확산	넓은 범위의 데이터를 빠르게 스캔
2010년대	Hadoop과 객체 저장소 기반 Data Lake의 확산	저비용 저장과 다양한 형식의 수용
2010년대 후반 이후	Lakehouse와 테이블 관리 기능의 결합	개방형 저장과 신뢰할 수 있는 테이블을 함께 제공

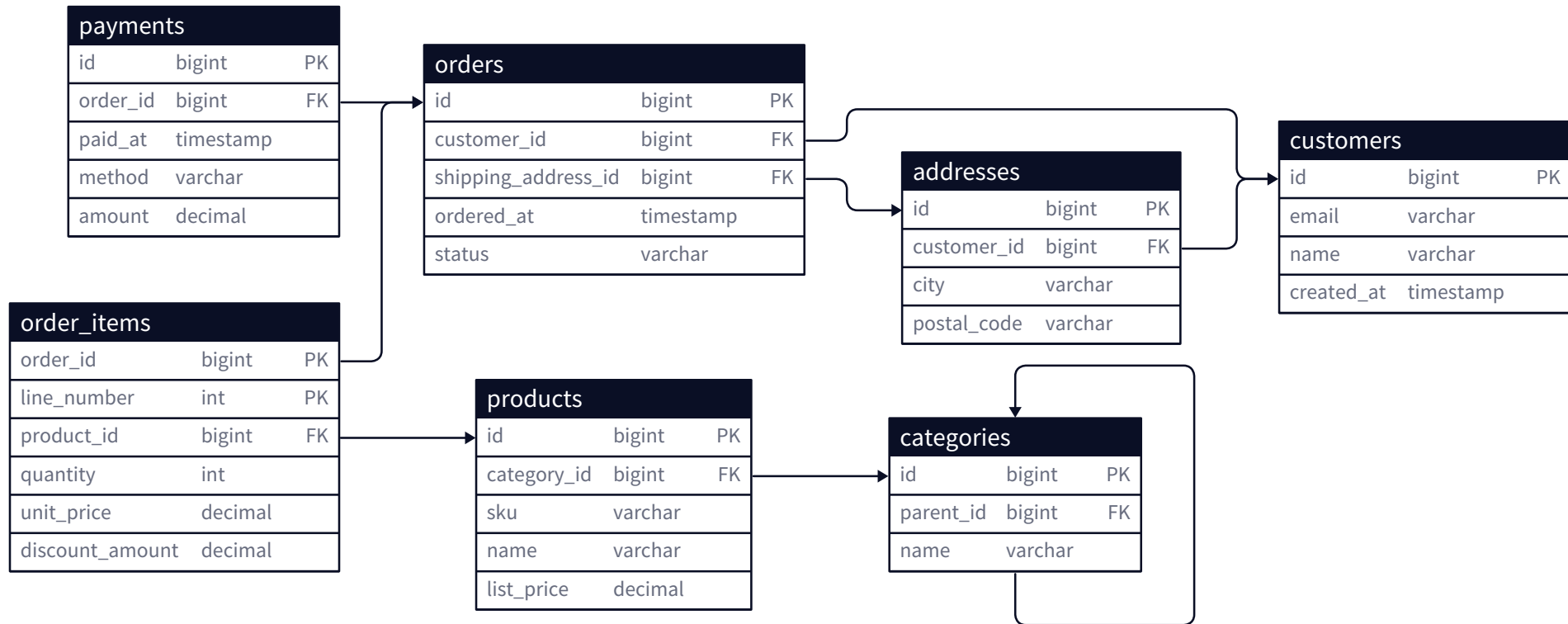
앞선 구조가 사라진 것이 아니라 서로 다른 목적의 계층으로 함께 남았습니다.

OLTP와 OLAP은 워크로드 스타일이다

관점	OLTP	OLAP
목적	현재 업무 상태를 정확히 변경합니다.	과거와 전체를 비교해 판단합니다.
대표 질문	이 주문을 결제할 수 있는가?	어느 지역의 분기 매출이 줄었는가?
접근 패턴	짧은 조회와 소량 쓰기가 빈번합니다.	넓은 스캔과 집계가 많습니다.
데이터 범위	최신 상태와 업무 처리 이력입니다.	여러 원천의 장기 이력입니다.
모델 경향	갱신 이상을 줄이는 정규화를 선호합니다.	탐색과 집계를 단순화하는 차원 모델을 자주 씁니다.
최적화 경향	인덱스, 동시성, 짧은 지연을 중시합니다.	컬럼 읽기, 압축, 병렬 스캔을 중시합니다.

OLTP와 OLAP은 제품 이름이나 특정 물리 스키마의 이름이 아닙니다.

출발점, 정규화된 커머스 OLTP



분석 질문은 운영 스키마를 해석하게 만든다

```
SELECT c.name, cat.name AS category,  
       SUM(oi.quantity * oi.unit_price - oi.discount_amount) AS sales  
FROM orders o  
JOIN order_items oi ON oi.order_id = o.id  
JOIN customers c ON c.id = o.customer_id  
JOIN products p ON p.id = oi.product_id  
JOIN categories cat ON cat.id = p.category_id  
WHERE o.status = 'PAID'  
GROUP BY c.name, cat.name;
```

- 질문마다 조인 경로와 업무 의미를 다시 해석합니다.
- 고객 지역과 상품 분류가 바뀌면 과거를 어떤 값으로 볼지도 결정해야 합니다.

운영 스키마가 잘못된 것이 아니라 분석 계약이 아직 없는 상태입니다.

Kimball의 네 가지 결정

순서	결정	이번 사례의 답
1	업무 프로세스를 선택합니다.	결제가 완료된 소매 주문 매출입니다.
2	그레인을 선언합니다.	주문 상품 한 줄입니다.
3	차원을 식별합니다.	날짜, 고객, 상품, 매장, 프로모션입니다.
4	팩트를 식별합니다.	수량, 총액, 할인액, 순매출액입니다.

차원과 팩트를 고르기 전에 한 행의 의미를 먼저 고정합니다.

출처: Kimball Group, [Four-Step Dimensional Design Process](#)

한 행의 계약을 먼저 쓴다

DECLARED GRAIN

팩트 테이블의 한 행은 주문 상품 한 줄을 나타냅니다.

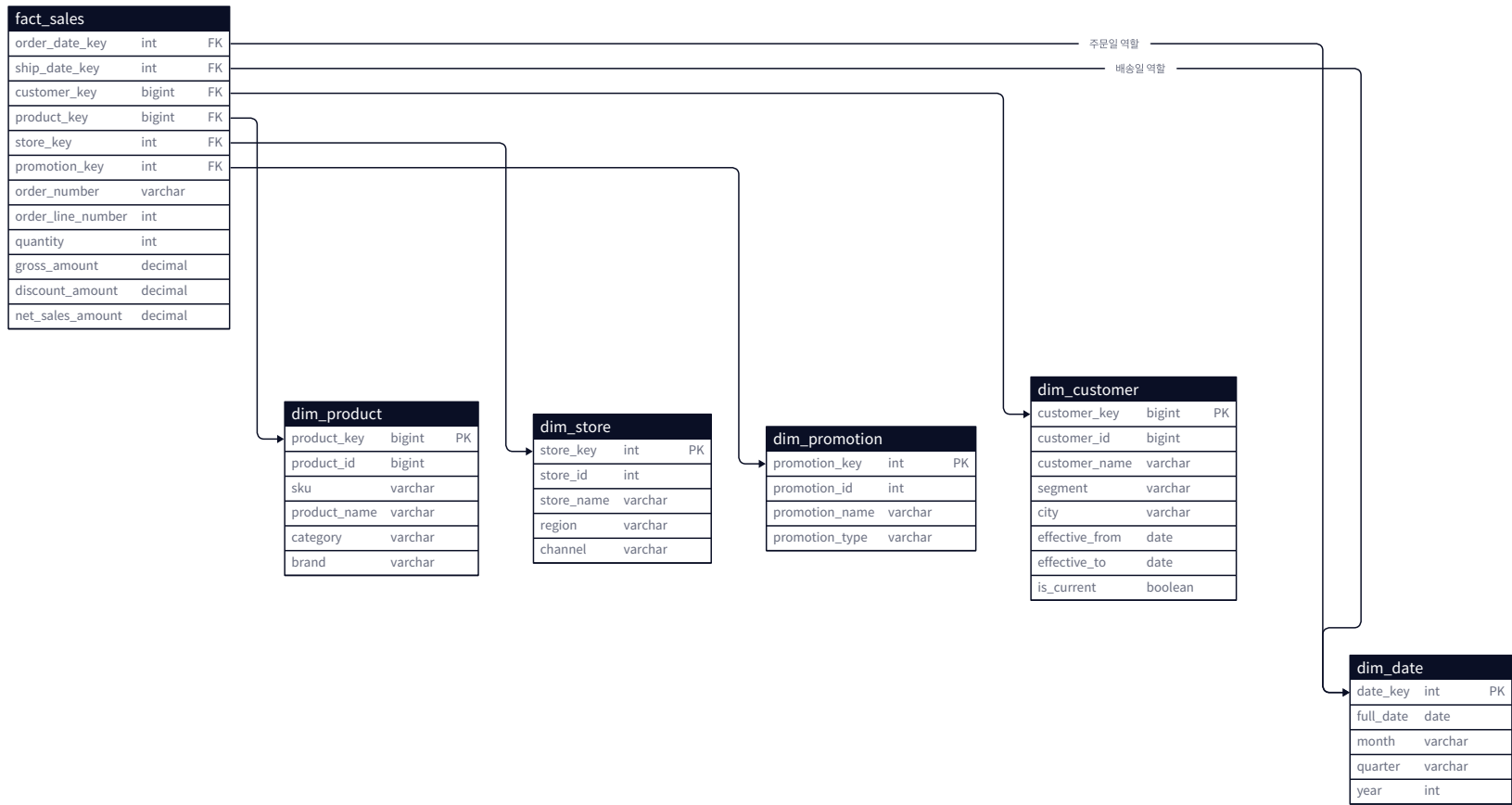
차원

- 날짜
- 고객
- 상품
- 매장과 채널
- 프로모션

팩트

- quantity
- gross_amount
- discount_amount
- net_sales_amount

결과, 소매 매출 Star Schema



중앙 팩트는 사건을 측정하고 주변 차원은 사건을 설명합니다. Star Schema는 BI가 아니라 분석용 데이터 모델입니다.

무엇이 바뀌었는가

관점	정규화된 운영 모델	Star Schema
한 행의 의미	각 업무 엔터티의 현재 상태입니다.	측정 사건의 원자 그레인입니다.
업무 문맥	여러 테이블을 따라가며 해석합니다.	차원 속성으로 필터링하고 그룹화합니다.
측정값	계산식이 보고서마다 흩어질 수 있습니다.	팩트의 정의와 가법성을 관리합니다.
시간	현재 상태 변경이 중심입니다.	당시의 문맥과 장기 이력을 보존합니다.
성능	짧은 트랜잭션에 최적화합니다.	별도의 물리 최적화가 추가로 필요합니다.

모델은 질문을 단순하게 만들지만, 성능을 자동으로 보장하지는 않습니다.

시간이 변한다, SCD Type 1과 Type 2

관점	SCD Type 1	SCD Type 2
변경 방식	기존 차원 행을 덮어씁니다.	새 대체 키로 행을 추가합니다.
과거 보고서	현재 속성으로 다시 보입니다.	당시 속성을 보존합니다.
필요한 열	현재 값이면 충분합니다.	시작일, 종료일, 현재 여부가 필요합니다.
적합한 예	오탈자 수정과 최신 분류입니다.	고객 등급과 영업 지역의 이력입니다.
비용	과거 문맥을 잃을 수 있습니다.	행 수와 ETL 복잡도가 늘어납니다.

모든 속성을 Type 2로 만들지 않고, 업무가 요구하는 이력만 보존합니다.

사건의 형태가 다르다

유형	한 행이 뜻하는 것	예시	기록 방식
트랜잭션 팩트	한 시점의 측정 사건입니다.	주문 상품 한 줄, 결제 한 건입니다.	사건마다 행을 추가합니다.
주기 스냅샷 팩트	고정된 기간의 상태나 활동입니다.	일별 재고, 월말 계좌 잔액입니다.	기간마다 행을 추가합니다.
누적 스냅샷 팩트	시작과 끝이 있는 프로세스입니다.	주문 접수부터 배송 완료까지입니다.	단계마다 같은 행을 갱신합니다.

같은 주문 도메인에서도 질문이 달라지면 팩트 테이블의 그레인이 달라집니다.

숫자를 함부로 더할 수 없다

가법성	의미	예시	안전한 집계
가산 팩트	모든 관련 차원에서 더할 수 있습니다.	판매 수량과 순매출액입니다.	합계를 구합니다.
준가산 팩트	일부 차원에는 더하지만 시간에는 어렵습니다.	일일 재고와 계좌 잔액입니다.	시점값이나 적절한 기간 평균을 사용합니다.
비가산 팩트	단순 합계가 의미를 잃습니다.	단가, 비율, 마진율입니다.	분자와 분모를 집계한 뒤 다시 계산합니다.

팩트마다 어떤 차원에서 어떤 연산이 유효한지 명시해야 합니다.

Conformed Dimension과 Bus Matrix

업무 프로세스	날짜	상품	매장	고객	프로모션
주문 매출	X	X	X	X	X
재고 스냅샷	X	X	X		
반품	X	X	X	X	
배송	X	X	X	X	

- 행은 독립된 팩트 테이블이 될 업무 프로세스입니다.
- 열은 여러 프로세스가 공유할 수 있는 차원입니다.
- 같은 열의 정의를 재사용하면 서로 다른 숫자를 같은 축에서 비교할 수 있습니다.

출처: Kimball Group, [Enterprise Data Warehouse Bus Matrix](#)

Drill-Across, 장부별 계산을 나란히 놓는다

판매 장부, 재고 장부, 반품 장부가 각각 따로 있다고 생각합니다.

상품	판매 장부	재고 장부	반품 장부
상품 A	3개 판매	10개 남음	1개 반품

1. 각 장부에서 상품 A의 값을 먼저 계산합니다.
2. 상품 A 라는 공통 기준으로 계산 결과를 한 줄에 놓습니다.

장부를 처음부터 직접 합치면 같은 재고가 여러 번 복사되어 계산이 틀릴 수 있습니다.

성능은 물리 계층에서 완성한다

Column Store

- 필요한 열만 읽습니다.
- 같은 타입의 값이 모여 압축에 유리합니다.
- 긴 스캔과 집계에 적합합니다.
- Star Schema와는 다른 층의 선택입니다.

Materialized Aggregate

- 자주 쓰는 그룹 합계를 미리 계산합니다.
- 저장 공간과 갱신 비용을 지불합니다.
- 반복되는 대화형 질의의 지연을 줄입니다.
- 질문 패턴이 바뀌면 다시 설계해야 합니다.

논리 모델은 무엇을 계산할지 정하고, 물리 설계는 어떻게 빠르게 계산할지 정합니다.

결론

운영의 한 줄과 분석의 한 줄은 목적이 다릅니다

**그레인을 고정하고, 시간과 프로세스를 통합한 뒤, 물리 계층을 최적화합
니다.**

Red Book 5판, The Data Warehouse Toolkit 3판, Kimball Group Dimensional Modeling Techniques

참고문헌

- Peter Bailis, Joseph M. Hellerstein, Michael Stonebraker 편집, [Readings in Database Systems, 5th Edition](#)
- Michael Stonebraker, [Chapter 1: Background](#)
- Ralph Kimball, Margy Ross, *The Data Warehouse Toolkit, 3rd Edition*, Wiley, 2013
- Kimball Group, [Dimensional Modeling Techniques](#)
- Oracle, [Data Warehousing Concepts](#)
- Bill Inmon, *Building the Data Warehouse*, Wiley, 1992
- D2, [SQL Tables](#)

스키마 다이어그램은 D2 0.7.1과 ELK 레이아웃으로 작성했습니다.